

A Vector Markov Model for Structural Coverage Growth and the Number of Failure Occurrences

Michael Grottke

Chair of Statistics and Econometrics, University of Erlangen-Nuremberg

Lange Gasse 20, D-90403 Nuernberg, Germany

Michael.Grottke@wiso.uni-erlangen.de

Abstract

Most software reliability growth models specify the expected number of failures experienced as a function of testing effort or calendar time. However, there are approaches to model the development of intermediate factors driving failure occurrences. This paper starts out with presenting a model framework consisting of four consecutive relationships. It is shown that a differential equation representing this framework is a generalization of several finite failures category models.

The relationships between the number of test cases executed and expected structural coverage, and between expected structural coverage and the expected number of failure occurrences are then explored further.

A vector Markov model allowing for partial redundancy in sampling code constructs is developed. The model bridges the gap between setups related to operational testing and systematic testing, respectively. Two extensions of the partial redundancy model considering the development of the number of failure occurrences are discussed.

The paper concludes with showing that the extended models fit into the structure of the differential equation presented at the beginning, which permits further interpretation.

1 A model framework

Most software reliability growth models specify the expected number of failure occurrences μ as a function of some measure of time. The authors of one of the earliest models, Jelinski and Moranda [6], for example, use calendar time t^* as the exogenous variable. However, they also state that an implicit assumption of their model is that the intensity of testing is constant over time. For cases in which this proposition does not hold, they formulate a refined model specifying two consecutive mappings: the one from calendar time

to testing effort t (which may be measured in man hours, CPU execution time, and the like), and the one from testing effort to the expected number of failure occurrences.

More recently, there have been efforts to identify intermediate factors driving the number of failure occurrences. Gokhale et al. [2] unify several finite failures category models of the Poisson type by considering structural coverage c of the application under test as such a driving factor. Structural coverage can be generically defined as the number of code constructs (for example, statements, blocks or paths) exercised by testing divided by the total number of code constructs in the tested software.

Rivers and Vouk [9, 10] specify the relationship between either test case coverage b - the percentage of planned test cases which have been executed - or structural coverage and the number of failure occurrences. They note that the number of failures experienced is not necessarily proportional to the coverage attained. Their solution is the introduction of an additional factor g , which they call testing efficiency.

Piwowski et al. [8] derive block coverage, a specific structural coverage, as a random variable C stochastically depending on the number of test cases executed i and discuss the expected structural coverage $\kappa(i) = E(C(i))$ and the mean value function $\mu(i)$, assuming that $\mu(i)$ and $\kappa(i)$ are proportional. If the total number of planned test cases, i_t , is known, both functions can be formulated with test case coverage $b = \frac{i}{i_t}$ being the exogenous variable.

Combining all these approaches, four consecutive relationships are identified:

- I. The allocation of testing effort t to calendar time t^*
- II. The development of test case coverage b as a function of cumulative testing effort
- III. The expected coverage of code constructs $\kappa = E(C)$ attained through test case coverage
- IV. The relationship between expected structural coverage and the expected number of failures experienced μ

Moreover, a model framework integrating the different concepts can be formulated as the following differential equation:

$$\frac{d\mu(\kappa(b(t(t^*))))}{dt^*} = \frac{dt(t^*)}{dt^*} \frac{d\kappa(b(t(t^*)))}{1 - \kappa(b(t(t^*)))} \times g(\kappa(b(t(t^*)))) [\nu_d - \mu(\kappa(b(t(t^*))))] \quad (1)$$

Under the classical assumption that as soon as a failure has occurred the fault which caused it is removed instantaneously and perfectly, the central proposition of this differential equation is that the failure intensity (the derivative of μ) depends on the expected number of detectable faults remaining in the software, $[\nu_d - \mu(\kappa(b(t(t^*))))]$. ν_d denotes the expected number of detectable faults present in the software at the beginning of testing. If the detection probability is the same time-invariant constant d for all faults, then $\nu_d = \nu \cdot d$. For software reliability models of the binomial type, ν stands for the fixed but unknown number of initial faults, u_0 . For models of the Poisson type, it represents the expected value N of the initial number of faults, which is assumed to follow a Poisson distribution.

The other factors in equation (1) are included in a specific model only if the respective relationships are in its scope. Furthermore, several relationships may be modeled together by merely specifying the product of the factors. This can be studied best by examining the models mentioned above as well as additional ones fitting in the model framework:

- The Goel-Okumoto model [1] starts out with a measure of testing effort. Therefore, the relationship I is not in its scope. Moreover, the relationships II, III and IV are not specified explicitly. It is only known that the product $\frac{d\kappa(t)/dt}{1-\kappa(t)}g(\kappa(t))$ is a constant value, say ϕ . A sufficient but not necessary condition is that both the hazard rate of the execution of a certain code construct and the testing efficiency take constant values. In the Goel-Okumoto model, the number of initial faults is a random variable following a Poisson distribution, and the detection probability is assumed to be equal to one. Consequently, in the specific differential equation ν_d is replaced by N . This yields

$$\frac{d\mu(t)}{dt} = \phi[N - \mu(t)]. \quad (2)$$

- Using the same propositions as the Goel-Okumoto model but assuming that the initial number of faults is fixed at the unknown value u_0 leads to the differential equation of the Jelinski-Moranda model [6],

$$\frac{d\mu(t)}{dt} = \phi[u_0 - \mu(t)].$$

- If relationship I is specified by some function $t(t^*) = W(t^*)$ and the product $\frac{d\kappa(t)/dt}{1-\kappa(t)}g(\kappa(t))$ is assumed to be a constant ϕ , then for $\nu = N$ and $d = 1$ the Goel-Okumoto model with a time-varying testing effort [14] is obtained:

$$\frac{d\mu(t^*)}{dt^*} = \frac{dW(t^*)}{dt^*} \phi[N - \mu(t^*)] \quad (3)$$

Note that one could as well transform the time scale by $t = W(t^*)$ and consider $\mu(t) = \mu(W(t^*))$ and $\frac{d\mu(t)}{dt} = \frac{d\mu(W(t^*))}{dW(t^*)}$, because the differential equation

$$\frac{d\mu(W(t^*))}{dW(t^*)} = \phi[N - \mu(W(t^*))]$$

can be written as

$$\frac{\frac{d\mu(W(t^*))}{dt^*}}{\frac{dW(t^*)}{dt^*}} = \frac{d\mu(W(t^*))}{dt^*} \frac{dt^*}{dW(t^*)} = \phi[N - \mu(W(t^*))],$$

which is equivalent to equation (3).

- Again, substituting u_0 for N in equation (3) yields a model of the binomial type, namely the refined Jelinski-Moranda model taking into account a time-varying testing effort [6]:

$$\frac{d\mu(t^*)}{dt^*} = \frac{dW(t^*)}{dt^*} \phi[u_0 - \mu(t^*)]$$

- The Enhanced Non-homogeneous Poisson Process (ENHPP) framework by Gokhale et al. [2] starts out with a measure of testing effort t . Structural coverage is supposed to be a deterministic function of this measure. This means that $\kappa(t) = E(C(t)) = c(t)$. Under the assumptions that the faults are distributed uniformly over the code constructs, that a fault causes a failure with constant probability d the first time that the construct at which it is located is exercised and that faults are removed instantaneously and perfectly as soon as they have caused a failure, the number of failure occurrences increases proportionally to structural coverage. Moreover, the per-fault hazard rate is equal to $\frac{dc(t)/dt}{1-c(t)}$, an expression which can be interpreted as the hazard rate of the execution of a certain code construct if $\lim_{t \rightarrow \infty} c(t) = 1$. This implies that the testing efficiency g is identical to one. Therefore, the differential equation of this model framework for non-homogeneous Poisson processes is

$$\frac{d\mu(t)}{dt} = \frac{dc(t)/dt}{1-c(t)} [Nd - \mu(t)].$$

The ENHPP framework itself includes a number of finite failures category models. For example, if $d = 1$

then a deterministic coverage function with a constant hazard rate ϕ leads to the Goel-Okumoto model (2), assuming a coverage hazard function of $\frac{\phi^2 t}{1+\phi t}$ results in the delayed S-shaped model [13], while for the coverage hazard function $\frac{\phi}{1+\psi \exp(-\phi t)}$ the inflection S-shaped model [7] is obtained. Remarkable is the new interpretation given by the ENHPP framework: The number of failure occurrences is proportional to structural coverage; a certain shape of the mean value function is caused by the fact that structural coverage as a function of testing effort has this shape.

- Rivers and Vouk [9, 10] treat test case coverage and structural coverage as equivalent and exchangeable, implying their numerical identity $c = c(b) = b$. The respective coverage measure is assumed to be the exogenous variable, i.e., the relationships I and II are not in the scope of the model. Moreover, as mentioned above, Rivers and Vouk introduce a factor g , whose value depends on current coverage, for taking into account that coverage growth and the development of the number of failures experienced are not necessarily proportional to each other. In the original formulation, the number of initial faults in the software is not specified to follow a Poisson distribution, but rather treated as an unknown, fixed value. The fault detection probability is constant at one. Therefore, the differential equation of the Rivers-Vouk model is

$$\frac{d\mu(b)}{db} = \frac{g(b)}{1-b} [u_0 - \mu(b)]$$

when starting out with test case coverage and

$$\frac{d\mu(c)}{dc} = \frac{g(c)}{1-c} [u_0 - \mu(c)].$$

when data on structural coverage is to be used.

These forms represent a meta model which needs further specification of the testing efficiency function. For example, setting $g(b) = \alpha(1-b)$ in the first case yields the so-called linear testing efficiency model, $\frac{d\mu(b)}{db} = \alpha[u_0 - \mu(b)]$. Obviously, its shape is similar to the one of the Jelinski-Moranda model; the main difference is that the exogenous variable is test case coverage, not testing effort.

- In the model by Piwowarski et al. [8] structural coverage growth is not deterministic like in the ENHPP framework, but a stochastic function of the number of test cases executed, i , or test case coverage, $b = \frac{i}{i_t}$. As a consequence, the mean value function is determined by the time-varying function of expected structural coverage, $\kappa(b) = E(C(b))$. The expected number of failure occurrences is assumed to be proportional to expected structural coverage (i.e., $g(b) \equiv 1$),

and all the u_0 faults in the software are thought to be detectable. Consequently, the differential equation of the model is given by

$$\frac{d\mu(b)}{db} = \frac{d\kappa(b)/db}{1-\kappa(b)} [u_0 - \mu(b)].$$

Figure 1 summarizes the structures of these models. Each relationship specified by a model is represented by an arrow between the two variables marked with bullets. If the endogenous variable of the relationship is assumed to be proportional or numerically identical to the exogenous variable, then this is indicated by the symbol $\propto$ or $=$ attached to the respective arrow.

Note that the intermediate variables of the models do not necessarily have to be measured. For example, even though the relationship between testing effort and structural coverage is modeled in the ENHPP framework, it is not estimated separately. Rather, only the resulting overall relationship between testing effort and the expected number of failure occurrences is fitted to the failure data collected during testing. The intermediate relationships are merely used as a means for deriving this overall model, but their individual adequacy is not checked.

The framework introduced here does not only unify a number of models in one generalizing equation. It also helps to structure assumptions according to the different consecutive relationships and to determine those propositions that have not been stated explicitly. Focusing on the intermediate relationships may help practitioners to better understand the shape of specific software reliability growth models. Moreover, the suggested forms of such relationships may be discussed with regard to their assumptions about the real world (e.g., the testing strategy used) and - if necessary - improved.

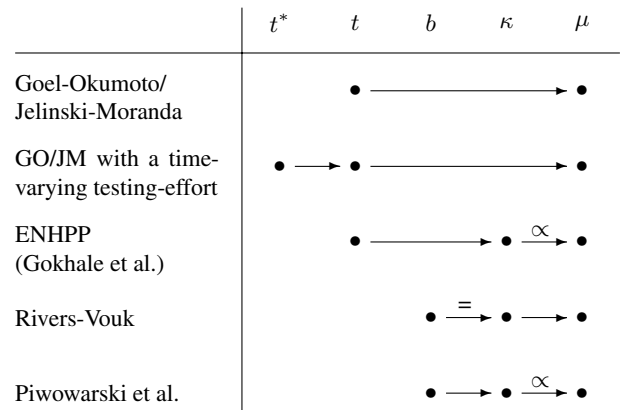


Figure 1. Relationships (implicitly) specified by different models; assumed equalities (=) and proportionalities ($\propto$) are indicated

In the remaining sections of this paper, the relationships between the number of test cases executed and structural coverage, and between structural coverage and the number of failure occurrences are explored further.

2 Modeling structural coverage growth

2.1 Piowarski et al. versus Rivers and Vouk

In both the approach by Piowarski et al. and the Rivers-Vouk model the relationship between test case coverage and structural coverage growth is contained.

Piowarski et al. [8] base their model on the following assumptions:

1. The program under test consists of G code constructs.
2. Per test case, p of these constructs are sensitized (i.e., executed) on average.
3. The p code constructs are always chosen from the entire population. The fact that a construct has already been sensitized does not diminish its chances of being sensitized in the future.

This setup resembles operational testing with a homogeneous operational profile, in which all non-overlapping and equally-sized operations have the same occurrence probability. Piowarski et al. show that the shape of expected structural coverage attained as a function of the number of test cases executed, $\kappa(i)$, is an exponential one, similar to the one of the Jelinski-Moranda model and the Goel-Okumoto model; i.e.,

$$\kappa(i) = 1 - \exp\left(-\frac{p}{G} \cdot i\right). \quad (4)$$

In figure 2, this equation is depicted as the solid line.

On the contrary, the model by Rivers and Vouk [9, 10] has explicitly been designed for systematic testing. An objective of this testing regime is to test as much of the application under test as possible. This implies that redundant executions of code constructs have to be avoided. While under the condition of equally-sized test cases the first two assumptions of the model by Piowarski et al. also apply, assumption 3 has to be replaced by the central proposition of the Rivers-Vouk model:

3. Code constructs which have already been exercised are not tested a second time.

Clearly, the gain in structural coverage is then $\frac{p}{G}$ for each of the equally-sized test cases, and

$$\kappa(i) = c(i) = \frac{p}{G} \cdot i. \quad (5)$$

This relationship is represented by the dotted straight line in figure 2.

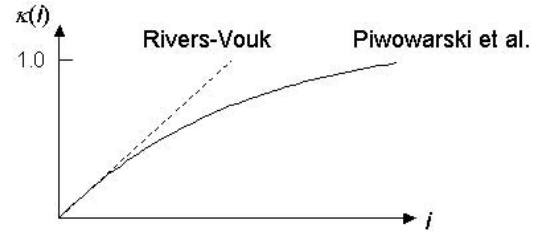


Figure 2. Structural coverage growth in the model by Piowarski et al. and in the Rivers-Vouk model

The two models mark extremes. Neither will testers re-execute the same portions of code over and over again during operational testing nor will testers following a systematic approach be able to perfectly avoid redundant executions of code constructs. A model allowing for partial redundancy could capture more realistic situations and bridge the gap between models for operational testing and those for systematic testing. In the following sections, variations of such a model are developed.

2.2 A vector Markov model for structural coverage growth

In order to consider partial redundancy in code construct execution, it seems useful to define three different states code constructs may take: “untested” (U), “already tested with the possibility of being tested again in the future” (T) and “tested and eliminated from further consideration” (E). The assumptions of the model are as follows:

1. The program under test consists of G code constructs. At the beginning of testing, all constructs are in state U .
2. Per test case, p constructs are sensitized on average.
3. The p constructs are randomly chosen from those constructs residing in state U or in state T at the beginning of the test case execution.
4. A constant fraction r ($0 \leq r \leq 1$) of those constructs exercised by a test case changes to (or stays in) state T and may be tested again in the future. The other constructs are eliminated and take state E .

This setup is a generalization of the one by Piowarski et al. and the one by Rivers and Vouk, because its assumptions 3 and 4 comprise both variations of proposition 3 in the two models.

It can be viewed as a discrete-time Markovian population model, more specific as a variation of the vector Markov process described by Howard [4]. While the state of the entire software is given by the number of untested, tested and eliminated code constructs, the behavior of each code construct is governed by the same Markov process with transitions between the states U , T and E taking place at the “atomic” test case executions. Let the probability that a code construct residing in state A before the i^{th} test case switches to state B during this test case be denoted by $\pi_{AB,i}$, with $A, B \in \{U, T, E\}$. A construct can only change its state when being sensitized during the test case execution. Therefore, the transition probabilities depend on the probability of being selected by the i^{th} test case. In accordance with assumption 2 for all constructs residing in state U or in state T at the beginning of the i^{th} test case execution this probability can be modeled as

$$\frac{p}{G_{U,i-1} + G_{T,i-1}},$$

with $G_{A,i-1}$ representing the number of constructs in state A ($A \in \{U, T, E\}$) after execution of the $(i-1)^{th}$ test case. We will denote this probability by z_i for the sake of brevity.

Since the fraction $(1-r)$ of the code constructs sensitized is eliminated, the transition probabilities $\pi_{UE,i}$ and $\pi_{TE,i}$ are both $z_i(1-r)$. A construct changes from state U to state T if it is exercised but not eliminated; therefore, $\pi_{UT,i} = z_i r$. The probability of remaining in state U , $\pi_{UU,i}$, is equal to $1 - \pi_{UT,i} - \pi_{UE,i} = 1 - z_i$, which is exactly the probability of not being selected. Likewise, $\pi_{TT,i} = 1 - \pi_{TE,i} = 1 - z_i(1-r)$, which can also be calculated as the probability of not being exercised plus the probability of being sensitized but not eliminated. The transition probability $\pi_{EE,i}$ is equal to one; state E is absorbing.

In figure 3 the structure of the resulting vector Markov process is depicted together with the transition probabilities for the i^{th} test case.

Obviously, z_i and the related transition probabilities are not only time-variant, they also depend on the condition of all code constructs. In contrast to the systems discussed in [4], the units belonging to the population are not acted upon independently.

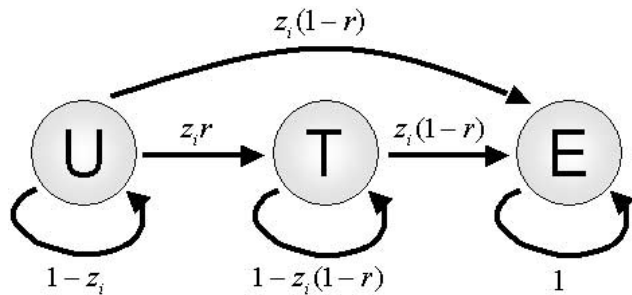


Figure 3. Transition diagram of the basic vector Markov model

Due to the structure of the model, it is possible to formulate the *expected* number of code constructs in the three states in a recursive way:

$$\begin{pmatrix} E(G_{U,i}) \\ E(G_{T,i}) \\ E(G_{E,i}) \end{pmatrix} = \begin{pmatrix} 1 - \zeta_i & 0 & 0 \\ \zeta_i r & 1 - \zeta_i(1-r) & 0 \\ \zeta_i(1-r) & \zeta_i(1-r) & 1 \end{pmatrix} \times \begin{pmatrix} E(G_{U,i-1}) \\ E(G_{T,i-1}) \\ E(G_{E,i-1}) \end{pmatrix} \quad (6)$$

ζ_i denotes the selection probability during the i^{th} test case, given the $(i-1)^{th}$ test case resulted in the expected allocation of the G code constructs to the three states, i.e.,

$$\zeta_i = \frac{p}{E(G_{U,i-1}) + E(G_{T,i-1})}.$$

Equation (6) and the following ones hold as long as it can be expected that at least p code constructs have not been eliminated so far, i.e., for $p \leq G - p(1-r)(i-1)$.

We are interested in the expected number of code constructs covered by the first i test cases, $E(Q_i)$, which is the sum of $E(G_{T,i})$ and $E(G_{E,i})$:

$$\begin{aligned} E(Q_i) &= \begin{pmatrix} 0 & 1 & 1 \end{pmatrix} \begin{pmatrix} E(G_{U,i}) \\ E(G_{T,i}) \\ E(G_{E,i}) \end{pmatrix} \\ &= \zeta_i E(G_{U,i-1}) + E(G_{T,i-1}) + E(G_{E,i-1}) \end{aligned} \quad (7)$$

Equations (6) and (7) could be used together with the initial conditions $E(G_{U,0}) = G_{U,0} = G$, $E(G_{T,0}) = G_{T,0} = 0$ and $E(G_{E,0}) = G_{E,0} = 0$, which follow from assumption 1, for recursively calculating the expected number of constructs in the different states and the number of code constructs sensitized after each test case. However, it is also possible to derive a (continuously approximated) closed form expression for the expected number of code constructs exercised, or for expected structural coverage.

Since $E(G_{T,i-1}) + E(G_{E,i-1}) = E(Q_{i-1})$, subtracting $E(G_{T,i-1}) + E(G_{E,i-1})$ from both sides of equation (7) yields

$$E(\Delta Q_i) = \zeta_i E(G_{U,i-1}). \quad (8)$$

It is now necessary to express both ζ_i and $E(G_{U,i-1})$ in terms of $E(Q_i)$. Obviously, the expected number of constructs not tested during the first $(i-1)$ test cases is exactly the number of all constructs minus the expected number of constructs that have already been sensitized: $E(G_{U,i-1}) = G - E(Q_{i-1})$. As for the denominator of ζ_i , $E(G_{U,i-1}) + E(G_{T,i-1})$, it is equal to $G - E(G_{E,i-1})$. Since a constant fraction $(1-r)$ of the p constructs executed by a test case is eliminated - regardless of the state these constructs were in

before -, the change in the number of eliminated constructs is always $p(1-r)$. Therefore, $E(G_{E,i-1})$ is nothing but the deterministic value $G_{E,i-1} = p(1-r)(i-1)$.

Consequently, equation (8) can be written as

$$E(\Delta Q_i) = p \cdot \frac{G - E(Q_{i-1})}{G - p(1-r)(i-1)} \quad (9)$$

or, dividing both sides by the total number of code constructs G , it can be formulated in terms of structural coverage:

$$E(\Delta C_i) = \frac{p}{G} \cdot \frac{1 - E(C_{i-1})}{1 - \frac{p}{G}(1-r)(i-1)} \quad (10)$$

It is possible to continuously approximate difference equation (10) by assuming that each test case can be split in any number of smaller test cases, i.e., by letting $\Delta i \rightarrow di \rightarrow 0$:

$$\frac{d\kappa(i)}{di} = \frac{p}{G} \cdot \frac{1 - \kappa(i)}{1 - \frac{p}{G}(1-r)i}$$

Integrating this differential equation finally yields

$$\kappa(i) = 1 - \left(1 - \frac{p}{G}(1-r)i\right)^{\frac{1}{1-r}} \quad (11)$$

for $0 \leq r < 1$ and $i \leq \left[\frac{p}{G}(1-r)\right]^{-1}$, and

$$\kappa(i) = 1 - \exp\left(-\frac{p}{G} \cdot i\right) \quad (12)$$

for $r = 1$.

These equations confirm that this vector Markov model is a generalization of the two approaches described in section 2.1: Equation (12) obtained for $r = 1$, i.e., if constructs are never eliminated, is identical to equation (4) derived by Piwowarski et al. Assuming perfect avoidance of redundancy ($r = 0$), on the other hand, leads to the linear relationship (5), which is connected to the setup proposed by Rivers and Vouk.

3 Vector Markov models for the number of failure occurrences

3.1 Common assumptions

The basic partial redundancy model derived in section 2.2 can be extended to include relationship IV of the model framework. This requires one to distinguish between correct and faulty code constructs. Like in the existing models explicitly specifying this relationship (e.g. the Rivers-Vouk model), an implicit assumption of this extended vector Markov model is that at most one fault can be located at a code construct and cause a failure. For coverage metrics partitioning the software in a large number of code constructs this proposition does not seem to be very restrictive.

In order to facilitate the distinction between correct and faulty code constructs, each of the states U , T and E is split into two: code constructs can be untested and faulty (UF), untested and correct (UC), tested and faulty (TF), tested and correct (TC), eliminated and faulty (EF) or eliminated and correct (EC).

This entails the necessity to restate the model assumptions:

1. The program under test consists of G code constructs. At the beginning of testing, u_0 of these constructs are in state UF (i.e., they are untested and faulty); the remaining $(G - u_0)$ constructs are in state UC .
2. Per test case, p constructs are sensitized on average.
3. The p constructs are randomly chosen from those constructs residing in one of the states UF , UC , TF or TC at the beginning of the test case execution.
4. A constant fraction r ($0 \leq r \leq 1$) of those constructs exercised by a test case may be tested again in the future. If such a construct is faulty after the test case execution, it changes to (or stays in) state TF ; if it is correct after the test case execution, it moves to (or remains in) state TC . The other constructs are eliminated and take state EF or state EC , respectively.

These rephrased assumptions lay the foundation for the model extension, but they do not change the way in which structural coverage grows.

As for the kind of development of the number of failures experienced, i.e. the structure of transitions from the states UF and TF to the states TC and EC , two variations are specified and discussed in the following subsections.

3.2 Fault detection at the first code construct execution only

As we have seen in section 1, several models explicitly specifying relationship IV assume that a failure may occur when a code construct at which a fault is located is exercised *for the first time*. A failure is then either caused with certainty (as in the approach by Piwowarski et al.) or - more generally - only with a certain fault detection probability d (as in the ENHPP framework).

Within the vector Markov model a proposition corresponding to this setup is as follows:

5. When a code construct at which a fault is located is exercised for the first time, the fault causes a failure with activation probability s ($0 < s \leq 1$). The fault is then removed instantaneously and perfectly. If no failure occurs during the first execution of the code construct, then the fault will not be detected until the end of testing.

Figure 4 represents the resulting transition diagram. Self loops have been omitted in order to maintain a less cluttered diagram. Moreover, the inverse probabilities $(1 - r)$ and $(1 - s)$ have been abbreviated by $\tilde{r}$ and $\tilde{s}$, respectively.

Since the way in which code constructs are sampled has not been changed, the transition probabilities between the “correct” states are similar to the corresponding ones in the basic model. Take a look at the transition probability from state UC to state TC , $\pi_{UC,TC,i}^{e_I}$, with the superscript e_I indicating that it is linked to the first extended model: $\pi_{UC,TC,i}^{e_I}$ is still the probability of being selected multiplied by the replacement rate r . However, when calculating the selection probability in the first extended model the number of constructs in one of the states UC , TC , UF and TF has to be considered, i.e.,

$$z_i^{e_I} = \frac{p}{G_{UC,i-1} + G_{TC,i-1} + G_{UF,i-1} + G_{TF,i-1}}.$$

Likewise, $\pi_{UCEC,i}^{e_I} = \pi_{TCEC,i}^{e_I} = z_i^{e_I}(1 - r)$.

As for the transitions from and between “faulty” states, the fault activation probability s comes into play. Since an untested, faulty construct switches to state TC if it is exercised but not eliminated and if a failure occurs, $\pi_{UF,TC,i}^{e_I} = z_i^{e_I}rs$. Similarly, $\pi_{UF,TF,i}^{e_I} = z_i^{e_I}r(1 - s)$ and $\pi_{TF,EF,i}^{e_I} = z_i^{e_I}(1 - r)(1 - s)$.

A faulty construct that has already been tested before can only remain in state TF or move to state EF , because according to assumption 5 the fault will never be detected and corrected. Therefore, $\pi_{TF,EF,i}^{e_I} = z_i^{e_I}(1 - r)$.

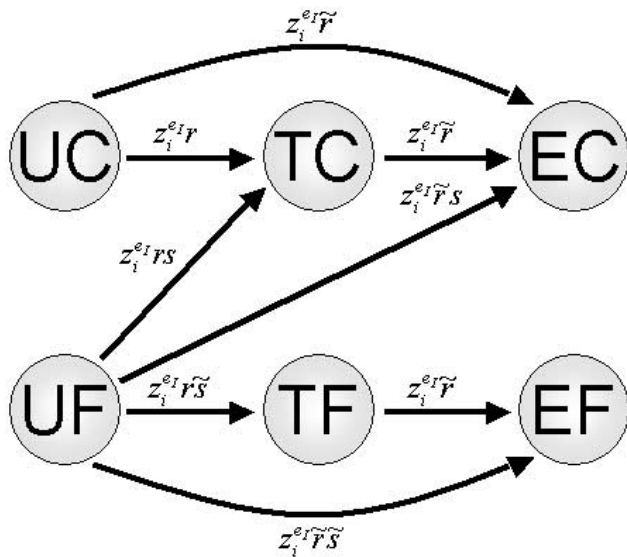


Figure 4. Transition diagram of the first extended vector Markov model

This leads to a recursive equation for the expected values of number of code constructs in the various states,

$$\begin{pmatrix} E(G_{UC,i}) \\ E(G_{UF,i}) \\ E(G_{TC,i}) \\ E(G_{TF,i}) \\ E(G_{EC,i}) \\ E(G_{EF,i}) \end{pmatrix} = \mathbf{P}_i^{e_I} \begin{pmatrix} E(G_{UC,i-1}) \\ E(G_{UF,i-1}) \\ E(G_{TC,i-1}) \\ E(G_{TF,i-1}) \\ E(G_{EC,i-1}) \\ E(G_{EF,i-1}) \end{pmatrix}, \quad (13)$$

which is valid for $p \leq G - p(1 - r)(i - 1)$. The matrix $\mathbf{P}_i^{e_I}$ is given by

$$\begin{pmatrix} 1 - \zeta_i^{e_I} & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 - \zeta_i^{e_I} & 0 & 0 & 0 & 0 \\ \zeta_i^{e_I}r & \zeta_i^{e_I}r\tilde{s} & 1 - \zeta_i^{e_I}\tilde{r} & 0 & 0 & 0 \\ 0 & \zeta_i^{e_I}r\tilde{s} & 0 & 1 - \zeta_i^{e_I}\tilde{r} & 0 & 0 \\ \zeta_i^{e_I}\tilde{r} & \zeta_i^{e_I}\tilde{r}s & \zeta_i^{e_I}\tilde{r} & 0 & 1 & 0 \\ 0 & \zeta_i^{e_I}\tilde{r}\tilde{s} & 0 & \zeta_i^{e_I}\tilde{r} & 0 & 1 \end{pmatrix},$$

with $\zeta_i^{e_I} = \frac{p}{E(G_{UC,i-1}) + E(G_{TC,i-1}) + E(G_{UF,i-1}) + E(G_{TF,i-1})}$.

The expected number of code constructs exercised by the first i test cases can be calculated as follows:

$$\begin{aligned} E(Q_i) &= \begin{pmatrix} 0 & 0 & 1 & 1 & 1 & 1 \end{pmatrix} \begin{pmatrix} E(G_{UC,i}) \\ E(G_{UF,i}) \\ E(G_{TC,i}) \\ E(G_{TF,i}) \\ E(G_{EC,i}) \\ E(G_{EF,i}) \end{pmatrix} \\ &= \zeta_i^{e_I} [E(G_{UC,i-1}) + E(G_{UF,i-1})] \\ &\quad + \sum_{A \in \{TC, TF, EC, EF\}} E(G_{A,i-1}) \end{aligned}$$

Since $\sum_{A \in \{TC, TF, EC, EF\}} E(G_{A,i-1}) = E(Q_{i-1})$,

$$E(\Delta Q_i) = \zeta_i^{e_I} [E(G_{UC,i-1}) + E(G_{UF,i-1})].$$

Like in the basic model, the denominator of $\zeta_i^{e_I}$ is equal to $G - p(1 - r)(i - 1)$. Moreover, the constructs in the states UC and UF are exactly those which have not been tested before; therefore, $E(G_{UC,i-1}) + E(G_{UF,i-1}) = G - E(Q_{i-1})$.

The difference equation derived for the expected number of code constructs covered is consequently

$$E(\Delta Q_i) = p \cdot \frac{G - E(Q_{i-1})}{G - p(1 - r)(i - 1)},$$

exactly the same as equation (9). This means that the continuously approximated functions for expected structural coverage derived with regard to the basic model - equations (11) and (12) - also apply to the first extended model.

This result is not really surprising, because when restating the model assumptions care has been given not to change the way in which coverage grows.

The expected number of faults still remaining in the software after execution of the i^{th} test case is given by

$E(G_{UF,i}) + E(G_{TF,i}) + E(G_{EF,i})$. Therefore, $E(M_i)$, the expected number of failure occurrences for the first i test cases, is

$$E(M_i) = u_0 - \begin{pmatrix} 0 & 1 & 0 & 1 & 0 & 1 \end{pmatrix} \begin{pmatrix} E(G_{UC,i}) \\ E(G_{UF,i}) \\ E(G_{TC,i}) \\ E(G_{TF,i}) \\ E(G_{EC,i}) \\ E(G_{EF,i}) \end{pmatrix}$$

$$= u_0 - (1 - \zeta_i^{e_I} s) E(G_{UF,i-1}) - E(G_{TF,i-1}) - E(G_{EF,i-1}).$$

Using the relationship $E(M_{i-1}) = u_0 - E(G_{UF,i-1}) - E(G_{TF,i-1}) - E(G_{EF,i-1})$, it follows:

$$\Delta E(M_i) = \zeta_i^{e_I} s E(G_{UF,i-1}) \quad (14)$$

This equation states the reason for growth in the number of failure occurrences in a quite intuitive way: Faulty constructs whose execution may cause failures are sensitized, and the fault is activated.

Since untested faulty and untested correct constructs have the same chance of being selected, $E(G_{UF,i-1})$ is always the same portion $\frac{u_0}{G}$ of the expected number of all untested constructs:

$$\begin{aligned} E(G_{UF,i-1}) &= \frac{u_0}{G} [E(G_{UC,i-1}) + E(G_{UF,i-1})] \\ &= \frac{u_0}{G} [G - E(Q_{i-1})] \\ &= u_0 [1 - E(C_{i-1})] \end{aligned} \quad (15)$$

For $r < 1$ and $i \leq \left[\frac{p}{G}(1-r)\right]^{-1}$, plugging equation (15) into equation (14) and letting Δi approach zero yields

$$\begin{aligned} d\mu(i) &= \frac{p}{G - p(1-r)i} s u_0 [1 - \kappa(i)] di \\ &= \frac{psu_0}{G} \left[1 - \frac{p}{G}(1-r)i\right]^{\frac{r}{1-r}} di. \end{aligned}$$

The continuously approximated function of the expected number of failure occurrences $\mu(i)$ is obtained by integrating this equation:

$$\begin{aligned} \mu(i) &= \frac{psu_0}{G} \int_0^i \left[1 - \frac{p}{G}(1-r)x\right]^{\frac{r}{1-r}} dx \\ &= u_0 s \left[1 - \left(1 - \frac{p}{G}(1-r)i\right)^{\frac{1}{1-r}}\right] \\ &= u_0 s \kappa(i) \end{aligned} \quad (16)$$

For $r = 1$, the continuous approximation of $\Delta E(M_i)$ is

$$\begin{aligned} d\mu(i) &= \frac{p}{G} s u_0 [1 - \kappa(i)] di \\ &= \frac{psu_0}{G} \exp\left(-\frac{p}{G}i\right) di, \end{aligned}$$

and the mean value function is given by

$$\begin{aligned} \mu(i) &= \frac{psu_0}{G} \int_0^i \exp\left(-\frac{p}{G}x\right) dx \\ &= u_0 s \left[1 - \exp\left(-\frac{p}{G}i\right)\right] \\ &= u_0 s \kappa(i). \end{aligned} \quad (17)$$

In the vector Markov model, the proposition that only the first execution of a faulty construct may produce a failure leads to a proportional relationship between expected structural coverage and the expected number of failure occurrences, just like in the ENHPP framework and in the approach by Piwowarski et al. The constant of proportionality is the expected number of detectable faults, $u_0 s$.

3.3 Fault detection at repeated code construct executions

It does not seem realistic to suppose that a fault can only be detected during the first execution of the construct at which it is located. Rather, repeated testing of a construct which is still faulty may finally reveal the fault.

To account for that, assumption 5 of the first extended model is replaced by the following one:

5. When a code construct at which a fault is located is exercised for the first time or repeatedly, the fault causes a failure with constant activation probability s ($0 < s \leq 1$). The fault is then removed instantaneously and perfectly.

This means that in the transition diagram of the second extended model (cf. figure 5) additional transitions from state TF to the states TC and EC have to be considered. The transition probabilities are equal to the corresponding ones starting from state UF , i.e., $\pi_{TF,TC,i}^{e_{II}} = \pi_{UF,TC,i}^{e_{II}} = z_i^{e_{II}} r s$ and $\pi_{TF,EC,i}^{e_{II}} = \pi_{UF,EC,i}^{e_{II}} = z_i^{e_{II}} \tilde{r} s$. At the same time, $\pi_{TF,TF,i}^{e_{II}}$ and $\pi_{TF,EF,i}^{e_{II}}$ are diminished. The superscript e_{II} indicates the expressions related to the second extended model. However, note that the selection probability is

$$z_i^{e_{II}} = \frac{p}{G_{UC,i-1} + G_{TC,i-1} + G_{UF,i-1} + G_{TF,i-1}},$$

just like in the first extended model.

Again, for $p \leq G - p(1-r)(i-1)$ the expected number of code constructs in the six states can be written recursively, analogously to equation (13), with matrix $\mathbf{P}_i^{e_{II}}$ (instead of $\mathbf{P}_i^{e_I}$) taking the form

$$\begin{pmatrix} 1 - \zeta_i^{e_{II}} & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 - \zeta_i^{e_{II}} & 0 & 0 & 0 & 0 \\ \zeta_i^{e_{II}} r & \zeta_i^{e_{II}} r s & 1 - \zeta_i^{e_{II}} \tilde{r} & \zeta_i^{e_{II}} r s & 0 & 0 \\ 0 & \zeta_i^{e_{II}} \tilde{r} s & 0 & 1 - \zeta_i^{e_{II}} (1 - r \tilde{s}) & 0 & 0 \\ \zeta_i^{e_{II}} \tilde{r} & \zeta_i^{e_{II}} \tilde{r} s & \zeta_i^{e_{II}} \tilde{r} & \zeta_i^{e_{II}} \tilde{r} s & 1 & 0 \\ 0 & \zeta_i^{e_{II}} \tilde{r} \tilde{s} & 0 & \zeta_i^{e_{II}} \tilde{r} \tilde{s} & 0 & 1 \end{pmatrix}.$$

First of all, the expected number of faults detected at full coverage according to the latter model variation, $u_0 \frac{s}{1-r+rs}$, is larger than the corresponding value in the former one, $u_0 s$. The difference is the larger the higher the redundancy level r is, because

$$\frac{d\left(u_0 \frac{s}{1-r+rs}\right)}{dr} = \frac{u_0 s(1-s)}{(1-r+rs)^2} > 0.$$

Secondly, the number of failure occurrences expected according to the second extended model is not proportional to the expected structural coverage achieved. Rather, it can be proved that the expected number of failures experienced per percentage of structural coverage gained increases. For $0 < r < 1$,

$$\begin{aligned} \frac{d\left(\frac{\mu(i)}{\kappa(i)}\right)}{di} &= \frac{u_0 s \frac{p}{G} (1 - \kappa(i))^{rs}}{(\kappa(i))^2} \\ &\times \left[1 - \frac{rs-r}{1-r+rs} (1 - \kappa(i)) - \frac{1}{1-r+rs} (1 - \kappa(i))^{r(1-s)} \right]. \end{aligned}$$

Both the fraction and the expression in brackets are always positive for $\kappa(i) \in (0, 1)$.

For $r = 1$,

$$\begin{aligned} \frac{d\left(\frac{\mu(i)}{\kappa(i)}\right)}{di} &= \frac{u_0 \frac{p}{G}}{(\kappa(i))^2} \left[s \exp\left(-\frac{p}{G} si\right) \right. \\ &\quad \left. + (1-s) \exp\left(-\frac{p}{G} (1+s)i\right) - \exp\left(-\frac{p}{G} i\right) \right]. \quad (23) \end{aligned}$$

There is no doubt about the positive sign of the fraction. The expression in brackets can be visualized in a graph showing the exponential function $\exp\left(-\frac{p}{G} x\right)$, where x is the exogenous variable (cf. figure 6). For any values $i > 0$ and $0 < s < 1$, the secant between the points $(si, \exp\left(-\frac{p}{G} si\right))$ and $((1+s)i, \exp\left(-\frac{p}{G} (1+s)i\right))$ can be drawn. Then $s \exp\left(-\frac{p}{G} si\right) + (1-s) \exp\left(-\frac{p}{G} (1+s)i\right)$ is exactly the y -value of that point on the secant for which x is equal to i .

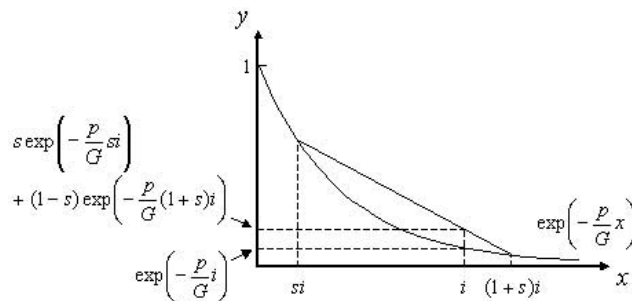


Figure 6. Graphical interpretation of the bracketed expression in equation (23)

Since the concave side of the exponential function points upward, this value is always larger than the y -value of the corresponding point on the curve, $\exp\left(-\frac{p}{G} i\right)$. Consequently, the bracketed expression and the derivative of $\frac{\mu(i)}{\kappa(i)}$ with respect to i are positive.

This means that regardless of the extent of redundancy $r > 0$, as long as the detection of faults is not certain the expected number of failure occurrences per percentage of structural coverage gained increases as testing proceeds according to the second extended model.

The reason for these properties of the second model variation lies in the fact that it allows for positive effects of redundancies in sampling code constructs: Faults in those constructs already covered before may still be detected.

4 Some remarks on parameter estimation

The focus of this paper is on the theoretical investigation of the structure of existing software reliability models and on the derivation of a failure model resulting from setups incorporating specific forms of operational and systematic testing. Of course, in order to facilitate the practical application of the variants of the partial redundancy model, approaches for estimating the model parameters have to be examined. While a complete discussion of this complex issue is not in the scope of this paper, at least some remarks are to be made here.

As for the data requirements, basically the model does not call for the collection of structural coverage information. Just as the ENHPP framework presented in section 1 the assumptions concerning code coverage growth are used as a means for deriving the overall model, but the intermediate relationships do not necessarily have to be estimated separately.

Mathematically, the partial redundancy model is related to the Rivers-Vouk model as well as to other models based on the hypergeometric distribution, like the model by Tohma et al. [5, 11, 12]. Therefore, the parameter estimation techniques suggested for these models may be applied here.

One of them is fitting the mean value function to the observed cumulative number of failure occurrences, m_i , via the least squares method [5, 9]. For both variations of the partial redundancy model the sum of squared errors (SSE) to be minimized has the same functional form:

$$\text{SSE} = \begin{cases} \sum_i \{m_i - \alpha [1 - \exp(-\beta i)]\}^2 & \text{if } \frac{p}{\beta} = 0 \\ \sum_i \left\{m_i - \alpha \left[1 - (1 - \varrho i)^{\frac{\beta}{\varrho}}\right]\right\}^2 & \text{if } 0 < \frac{p}{\beta} \leq 1, \\ & i < \varrho^{-1} \end{cases}$$

Depending on the model variant, the parameters α , β and ϱ represent different functions of the original parameters G , p , r , u_0 and s (cf. table 1).

Table 1. Interpretation of the identifiable parameters

	First extended model	Second extended model
α	$u_0 s$	$u_0 \frac{s}{1-r+rs}$
β	$\frac{p}{G}$	$\frac{p}{G}(1-r+rs)$
ϱ	$\frac{p}{G}(1-r)$	$\frac{p}{G}(1-r)$

Two conclusions can be drawn: Firstly, applying the least squares method to the cumulative failure data, only the parameters α , β and ϱ can be estimated, whereas it is not possible to identify all of the parameters G , p , r , u_0 and s . Moreover, based on this procedure the first and the second extended model cannot be distinguished.

Telling the model variants apart is possible, however, if structural coverage information has been collected (by employing a coverage measurement tool), because then in a first step r and $\frac{p}{G}$ can be estimated from equations (11) and (12) via the least squares approach before fitting the mean value functions to the failure data.

There are several statistical problems connected to the fact that each random variable M_i is the sum of the number of failure occurrences within the first $(i-1)$ test cases, M_{i-1} , and the number of failures experienced during the i^{th} test case, ΔM_i . First of all, the disturbances in the model $M_i = \mu(i) + \varepsilon_i$ are heteroscedastic and correlated, which results in unbiased but inefficient parameter estimators. Moreover, since both i and m_i are trended series the significance of the relationship between the two series tends to be overestimated [3].

It has therefore been suggested to minimize the sum of the squared deviations of each Δm_i from its conditional expected value $E(\Delta M_i \mid \Delta m_1, \dots, \Delta m_{i-1})$ instead [12]. Since a large number of test cases without any failure occurrence can distort the estimates, such test cases should be grouped with test cases for which Δm_i is larger than zero.

The third approach that can be applied is maximum likelihood estimation [12]. For this end, the model assumptions have to be formulated slightly more stringent (e.g., by proposing that *exactly* p code constructs are sensitized per test case). The resulting likelihoods are then functions of (conditional) hypergeometric and binomial distributions. Replacing the binomial coefficients with combinations of gamma functions, these likelihoods can then be continuously approximated and maximized with respect to the model parameters.

The details of parameter estimation for the partial redundancy model and its application to data collected at software development and testing companies are the subject of the current research activities.

5 The vector Markov models and the model framework

It remains to be examined whether the functions derived for the two extended partial redundancy models fit into the differential equation (1) presented at the beginning of this paper, or whether this equation has to be generalized further.

In sections 2.2 and 3, expected structural coverage κ and the expected number of failure occurrences μ have been formulated in terms of the absolute number of test cases executed. They can easily be restated as functions of test case coverage $b = \frac{i}{i_t}$, with i_t denoting the total number of test cases planned. The following equations are valid for $b \leq [\frac{p}{G}(1-r)i_t]^{-1}$.

For both extended models, expected structural coverage is the same:

$$\kappa(b) = \begin{cases} 1 - (1 - \frac{p}{G}(1-r)i_t b)^{\frac{1}{1-r}} & \text{if } 0 \leq r < 1 \\ 1 - \exp(-\frac{p}{G}i_t b) & \text{if } r = 1 \end{cases}$$

In the first extended model, in which failures may only occur at the first execution of a faulty construct, the expected number of failures experienced in terms of test case coverage is

$$\mu(b) = \begin{cases} u_0 s \left[1 - (1 - \frac{p}{G}(1-r)i_t b)^{\frac{1}{1-r}} \right] & \text{if } 0 \leq r < 1 \\ u_0 s \left[1 - \exp(-\frac{p}{G}i_t b) \right] & \text{if } r = 1 \end{cases}$$

It can easily be shown that for both cases the following differential equation holds:

$$\frac{d\mu(b)}{db} = \frac{d\kappa(b)/db}{1 - \kappa(b)} [u_0 s - \mu(b)]$$

This equation is a special case of equation (1). This formulation confirms two properties of the model already discussed in section 3.2: Firstly, $\nu_d = u_0 s$ is the number of faults expected to be detected when full structural coverage has been attained. Moreover, since the factor g present in equation (1) is identical to one here, the expected number of failure occurrences develops proportionally to structural coverage.

The mean value function of the second extended model in terms of test case coverage b is:

$$\mu(b) = \begin{cases} u_0 \frac{s}{1-r+rs} \times \left[1 - (1 - \frac{p}{G}(1-r)i_t b)^{\frac{1-r+rs}{1-r}} \right] & \text{if } 0 \leq r < 1 \\ u_0 \left[1 - \exp(-\frac{p}{G}i_t b) \right] & \text{if } r = 1 \end{cases}$$

Again, the differential equation being valid for both cases of this model,

$$\frac{d\mu(b)}{db} = \frac{d\kappa(b)/db}{1 - \kappa(b)} (1-r+rs) \left[u_0 \frac{s}{1-r+rs} - \mu(b) \right],$$

is a special case of equation (1). As already noted before in section 3.3, the expected number of detectable faults ν_d is $u_0 \frac{s}{1-r+rs}$. This means that the detection probability d is

given by $\frac{s}{1-r+rs}$. The equation additionally reveals that the second extended model implies a constant testing efficiency of $g = 1 - r + rs$, which is smaller than one for $r > 0$ and $s < 1$. Interestingly, the value of the testing efficiency is equal to the denominator of the detection probability. Therefore, there seems to exist a connection between testing efficiency and the number of detectable faults not discussed in literature so far.

6 Conclusions

The model framework presented in this paper is useful for systematically examining existing models as well as for deriving new ones. Generalizing the relationship between test case coverage and structural coverage as specified in the model by Piwowarski et al. and in the Rivers-Vouk model, respectively, the basic partial redundancy model bridges the gap between models for data collected during operational testing and those intended for data collected during systematic testing. Moreover, the model can be extended in various ways in order to consider the expected number of failure occurrences. While the first variation discussed implies a constant testing efficiency of one (and therefore proportionality between structural coverage and the number of failures experienced), the relationship between structural coverage and the number of failure occurrences derived for the second variation typically involves a constant testing efficiency smaller than one.

Instead of constant values for r and s , time-varying probabilities could be examined as well. Even for complicated assumptions, the expected structural coverage and the mean value function in terms of the number of test cases executed can easily be calculated recursively. However, the derivation of closed-form expressions may not be possible then.

Besides the details of parameter estimation, future work will be concerned with the probability mass functions of the number of code constructs covered and the number of failure occurrences after i test case executions.

Acknowledgements

The research described in this paper was done in the course of the project PETS. This project is supported by the European Community in the framework of the specific program for research, technological development and demonstration on a user friendly society (1998-2002), the "IST Program". The author is solely responsible for this paper. It does not represent the opinion of the Community.

References

- [1] A. L. Goel and K. Okumoto. Time-dependent error-detection rate model for software reliability and other performance measures. *IEEE Trans. Reliability*, 28:206–211, 1979.
- [2] S. S. Gokhale, T. Philip, P. Marinos, and K. S. Trivedi. Unification of finite failure non-homogeneous Poisson process models through test coverage. In *Proc. Seventh International Symposium on Software Reliability Engineering*, pages 299–307, 1996.
- [3] W. H. Greene. *Econometric Analysis*. Prentice-Hall, Upper Saddle River, 3rd edition, 1997.
- [4] R. A. Howard. *Dynamic Probabilistic Systems - Volume I: Markov Models*. Series in decision and control. Wiley, New York, 1971.
- [5] R. Jacoby and Y. Tohma. Parameter value computation by least square method and evaluation of software availability and reliability at service-operation by the hyper-geometric distribution software reliability growth model (HGDM). In *Proc. Thirteenth International Conference on Software Engineering*, pages 226–237, 1991.
- [6] Z. Jelinski and P. Moranda. Software reliability research. In W. Freiberger, editor, *Statistical Computer Performance Evaluation*, pages 465–484. Academic Press, New York, 1972.
- [7] M. Ohba. Software reliability analysis models. *IBM Journal of Research and Development*, 28:428–443, 1984.
- [8] P. Piwowarski, M. Ohba, and J. Caruso. Coverage measurement experience during function test. In *Proc. Fifteenth International Conference on Software Engineering*, pages 287–301, 1993.
- [9] A. T. Rivers. *Modeling Software Reliability During Non-Operational Testing*. PhD thesis, North Carolina State University, 1998.
- [10] A. T. Rivers and M. A. Vouk. Resource-constrained non-operational testing of software. In *Proc. Ninth International Symposium on Software Reliability Engineering*, pages 154–163, 1998.
- [11] Y. Tohma, K. Tokunaga, S. Nagase, and Y. Murata. Structural approach to the estimation of the number of residual software faults based on the hyper-geometric distribution. *IEEE Trans. Software Engineering*, 15:345–355, 1989.
- [12] Y. Tohma, H. Yamano, M. Ohba, and R. Jacoby. The estimation of parameters of the hypergeometric distribution and its application to the software reliability growth model. *IEEE Trans. Software Engineering*, 17:483–489, 1991.
- [13] S. Yamada, M. Ohba, and S. Osaki. S-shaped software reliability growth modeling for software error detection. *IEEE Trans. Reliability*, 32:475–478, 1983.
- [14] S. Yamada, H. Ohtera, and H. Narihisa. Software reliability growth models with testing-effort. *IEEE Trans. Reliability*, 35:19–23, 1986.